APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes integrados de teoría y prácticas

October 21, 2026

Contents

1 Fundamentos del aprendizaje automático y modelos clásicos **2**

1.1 Apuntes del Tema 1: Foundations of Machine Learning — Classical Models & Clustering . . . 2

1.1.1 1. Conceptos Fundamentales y Paradigmas de Aprendizaje . . . 2

1.1.2 2. k-Nearest Neighbors (kNN) . . . 3

1.1.3 3. Naive Bayes . . . 4

1.1.4 4. Árboles de Decisión (Decision Trees) . . . 4

1.1.5 5. Clustering (Agrupamiento No Supervisado) . . . 5

1.1.6 6. Práctica y Aplicación en Código . . . 6

2 Ensembles y descomposición multiclase **8**

2.1 Apuntes del Tema 2: Ensemble Models & Multiclass Decomposition . . . 8

2.1.1 1. Motivación y Métodos de Ensamble (*Ensembles*) . . . 8

2.1.2 2. Bagging y Random Forest . . . 8

2.1.3 3. Boosting: AdaBoost y Gradient Boosting (XGBoost) . . . 9

2.1.4 4. Descomposición Multiclase: OVA y OVO . . . 10

2.1.5 5. Práctica y Aplicación en Código (Scikit-Learn) . . . 10

3 Regresión lineal **13**

3.1 Apuntes del Tema 3: Regresión Lineal . . . 13

3.1.1 1. Introducción y Planteamiento del Modelo . . . 13

3.1.2 2. Función de Pérdida (MSE) . . . 13

3.1.3 3. Optimización . . . 13

3.1.4 4. Escalado y Expansión Polinomial . . . 13

3.2 Conexión con la práctica . . . 13

3.2.1 Fuente: `03_01_Linear_regression.ipynb` . . . 13

4 Regresión logística **14**

4.1 Apuntes del Tema 4: Regresión Logística . . . 14

4.1.1 1. Concepto y Función Sigmoide . . . 14

4.1.2 2. Función de Pérdida (BCE) . . . 14

4.1.3 3. Regularización (L_1 y L_2) . . . 14
4.1.4 4. Práctica . . . 14
4.2 Autoevaluación . . . 14

5 Perceptrones multicapa **15**
5.1 Apuntes del Tema 5: Multilayer Perceptron (MLP I y II) . . . 15
5.1.1 1. De Modelos Lineales a No Lineales . . . 15
5.1.2 2. Capas Ocultas y Funciones de Activación . . . 15
5.1.3 3. Clasificación Multiclase: Regresión Softmax . . . 15
5.1.4 4. Algoritmo de Propagación: Forward Pass y Backpropagation . . . 16
5.1.5 5. Propiedades Teóricas e Inicialización de Parámetros . . . 17
5.1.6 6. Problemas en Redes Profundas: Desvanecimiento y Explosión del Gradiente . . . 17
5.1.7 7. Práctica y Aplicación en Código (PyTorch) . . . 18
5.2 Conexión con las prácticas . . . 19
5.3 Autoevaluación razonada . . . 19

6 Regularización en redes profundas **20**
6.1 Apuntes del Tema 6: Regularización en Redes Profundas . . . 20
6.1.1 1. El Problema del Sobreajuste (*Overfitting*) en Redes Profundas . . . 20
6.1.2 2. Decaimiento de Pesos (*Weight Decay* / Regularización L_2 vs. L_1) . . . 20
6.1.3 3. Dropout (Inyección de Ruido en Activaciones) . . . 21
6.1.4 4. Parada Temprana (*Early Stopping*) . . . 22
6.1.5 5. Práctica y Aplicación en Código (PyTorch) . . . 22
6.2 Conexión con las prácticas . . . 24
6.3 Autoevaluación razonada . . . 24

7 Descenso de gradiente y optimizadores **25**
7.1 Apuntes del Tema 8: Descenso de Gradiente, Métodos Acelerados y Adaptativos 25
7.1.1 1. Fundamentos de Optimización en Deep Learning . . . 25
7.1.2 2. Algoritmos de Descenso de Gradiente . . . 26
7.1.3 3. Optimizadores Acelerados y Adaptativos . . . 27
7.1.4 4. Políticas de Ajuste (*Learning Rate Schedulers*) y Warmup . . . 28
7.1.5 5. Práctica y Aplicación en Código (PyTorch) . . . 28
7.2 Conexión con las prácticas . . . 29
7.3 Autoevaluación razonada . . . 29

8 Redes neuronales convolucionales **31**
8.1 Apuntes del Tema 9: Redes Neuronales Convolucionales (CNN I y II) . . . 31
8.1.1 1. De MLPs a Convoluciones: Motivación e Sesgos Inductivos . . . 31

8.1.2 2. Operaciones Convolucionales y sus Parámetros 31
8.1.3 3. Capas de Agregación (*Pooling*) 32
8.1.4 4. Jerarquía de Representaciones y Arquitecturas Clásicas 32
8.1.5 5. Gestión de Datos en PyTorch: `Dataset` y `DataLoader` 33
8.1.6 6. Práctica y Aplicación en Código (PyTorch y Torchvision) 33
8.2 Conexión con las prácticas 35
8.3 Autoevaluación razonada 35

9 Datos secuenciales y redes recurrentes 36
9.1 Apuntes del Tema 10: Redes Neuronales Recurrentes 36
9.1.1 1. De Datos Estáticos a Datos Secuenciales 36
9.1.2 2. Arquitectura de la Red Neuronal Recurrente Básica (RNN Vanilla) . 36
9.1.3 3. Entrenamiento: Retropropagación a través del Tiempo (BPTT) . . . 37
9.1.4 4. Redes Gated: LSTM y GRU 37
9.1.5 5. RNNs Bidireccionales y Apiladas (*Deep RNNs*) 38
9.1.6 6. Práctica y Aplicación en Código (PyTorch) 39
9.2 Conexión con las prácticas 40
9.3 Autoevaluación razonada 40

10 Atención y Transformers 41
10.1 Apuntes del Tema 11: Attention Mechanisms and Transformers 41
10.1.1 1. Introducción y Limitaciones de los Modelos Secuenciales (Seq2Seq) . 41
10.1.2 2. Mecanismos de Atención en RNNs (Bahdanau et al., 2014) 41
10.1.3 3. Autoatención (*Self-Attention*) en Transformers 42
10.1.4 4. Atención Multicabeza (*Multi-Head Self-Attention*) 43
10.1.5 5. Arquitectura del Transformer: Encoder 43
10.1.6 6. Arquitectura del Transformer: Decoder 44
10.1.7 7. Consejos de Entrenamiento, Inferencia y Arquitecturas 45
10.1.8 8. Práctica en Código (PyTorch) 46
10.2 Conexión con las prácticas 47
10.3 Autoevaluación razonada 47

11 Sistemas de recomendación 48
11.1 Conexión con las prácticas 48
11.1.1 Fuente práctica: `13_01_Recommender_systems.ipynb` 48
11.2 Autoevaluación razonada 49

1 Fundamentos del aprendizaje automático y modelos clásicos

1.1 Apuntes del Tema 1: Foundations of Machine Learning — Classical Models & Clustering

1.1.1 1. Conceptos Fundamentales y Paradigmas de Aprendizaje

1.1.1.1 1.1 Trazabilidad y Definición

- **Machine Learning:** Disciplina enfocada en la creación de sistemas capaces de realizar tareas mediante el reconocimiento de patrones y la toma de decisiones basadas en datos de experiencia, evaluando el desempeño mediante la capacidad de **generalización** hacia nuevos datos no vistos y previniendo el *overfitting*.
- **Estructura de Datos Básica:**
  - **Dataset:** Conjunto de observaciones $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$.
  - **Instancia (x):** Vector de características de dimensión d, donde $x \in \mathbb{R}^d$.
  - **Etiqueta/Target (y):** Valor objetivo o clase a predecir.
  - **Modelo Paramétrico:** Función de mapeo $f(x;\theta) \rightarrow \hat{y}$ parametrizada por pesos y sesgos $\theta = (w, b)$.

1.1.1.2 1.2 Paradigmas de Aprendizaje

- **Supervisado:** Aprendizaje mediante pares entrada-salida etiquetados (x, y). Se divide en:
  - **Regresión:** El target y es un valor continuo.
  - **Clasificación:** El target y representa una categoría discreta.
- **No Supervisado:** Descubrimiento de estructuras implícitas o agrupamientos en datos sin etiquetas x (ej. *Clustering*, reducción de dimensionalidad).
- **Otros Paradigmas Relevantes:**
  - *Semi-supervised*: Combina una pequeña cantidad de datos etiquetados con una gran cantidad de datos no etiquetados para mejorar la generalización.
  - *Self-supervised*: Aprendizaje donde el propio modelo genera las etiquetas a partir de los datos mediante tareas pretexto (*pretext tasks*) para capturar representaciones significativas.

- *Transfer Learning*: Reutilización de modelos o backbones previamente entrenados en otra tarea o dominio para reducir la necesidad de datos.
- *Active Learning*: Estrategia en la que el modelo interactúa seleccionando los puntos de datos más informativos para que un humano los etiquete, reduciendo costes.
- *Online/Incremental Learning*: Actualización continua del modelo en tiempo real a medida que fluyen nuevos datos, útil para entornos dinámicos.
- *Federated Learning*: Entrenamiento colaborativo de un modelo global de forma distribuida en dispositivos locales sin compartir los datos privados de los usuarios.

1.1.2 2. k-Nearest Neighbors (kNN)

1.1.2.1 2.1 Principio e Intuición

- **Algoritmo Lazy Learner:** No requiere una fase de entrenamiento o ajuste de parámetros explícito ($f(x)$ se evalúa en tiempo de consulta).
- **Regla de Clasificación:** Asigna la etiqueta a una muestra de prueba x_{query} según el voto mayoritario (o promedio ponderado por la distancia inversa) de sus k vecinos más cercanos en el espacio métrico.

1.1.2.2 2.2 Métricas de Distancia

- **Distancia Euclídea (L_2):**

$$\text{dist}_{\text{Euclidean}}(x, x') = \sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2}$$

- **Distancia Manhattan (L_1):**

$$\text{dist}_{\text{Manhattan}}(x, x') = \sum_{j=1}^{d}|x_j - x'_j|$$

- **Minkowski (L_p):**

$$\text{dist}_{L^p}(x, x') = \left(\sum_{j=1}^{d}|x_j - x'_j|^p\right)^{1/p}$$

1.1.2.3 2.3 Selección de k y Efectos de Escala

- **Efecto de k:**
  - $k = 1$ (Bajo): Frontera extremadamente flexible, muy sensible al ruido y propenso a sobreajuste (*overfitting*).
  - k elevado: Fronteras muy suaves, riesgo de subajuste (*underfitting*) al mezclas vecindarios.
- **Necesidad de Normalización:** Dado que kNN depende de distancias geométricas, las variables con rangos numéricos elevados dominan el cálculo. Es indispensable estandarizar las características ($x' = \frac{x-\mu}{\sigma}$) antes de calcular las distancias.

- **La Maldición de la Dimensionalidad (*Curse of Dimensionality*):** A medida que la dimensión d del espacio crece, el volumen aumenta exponencialmente, volviendo los datos muy dispersos y reduciendo la capacidad discriminativa de las distancias geométricas entre los puntos.

1.1.3 3. Naive Bayes

1.1.3.1 3.1 Fundamento Probabilístico Generativo

- **Modelo Generativo:** Modela la probabilidad de la clase a priori $P(y)$ y la verosimilitud de los datos $P(x|y)$ para estimar la probabilidad a posteriori $P(y|x)$ aplicando el Teorema de Bayes:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)} \propto P(y) \prod_{j=1}^{d} P(x_j|y)$$

- **Regla MAP (Maximum A Posteriori):** Selección de la clase que maximiza el producto:

$$\hat{y} = \arg\max_{y} P(y) \prod_{j=1}^{d} P(x_j|y)$$

- **Asunción "Naive" (Ingenua):** Presupone la **independencia condicional** de todas las características x_j dada la clase y.

1.1.3.2 3.2 Variantes Principales de Naive Bayes

Variante	Tipo de Variable (x_j)	Estimación de Verosimilitud $P(x_j\|y)$
Categorical NB	Categóricas/Nominales discretas	$P(x_j = v \mid y) = \frac{\text{count}(x_j=v,y)+\alpha}{\text{count}(y)+\alpha \cdot K_j}$
Multinomial NB	Conteos discretos (frecuencias de texto)	$P(x_j \mid y) = \frac{\text{count}(x_j \text{ en clase } y)+\alpha}{\sum_k \text{count}(x_k \text{ en clase } y)+\alpha \cdot V}$
Gaussian NB	Características continuas	$P(x_j \mid y) = \frac{1}{\sqrt{2\pi\sigma_{j,y}^2}} \exp\left(-\frac{(x_j-\mu_{j,y})^2}{2\sigma_{j,y}^2}\right)$

- **Laplace Smoothing ($\alpha = 1$):** Corrige el problema de la **probabilidad cero** sumando el parámetro α en el numerador y $\alpha \cdot K_j$ (o $\alpha \cdot V$) en el denominador.

1.1.4 4. Árboles de Decisión (Decision Trees)

1.1.4.1 4.1 Concepto y Particionado Recursivo

- Modelos no paramétricos e interpretables para clasificación y regresión que dividen recursivamente el espacio de características mediante reglas ortogonales a los ejes.

1.1.4.2 4.2 Comparativa de Algoritmos y Criterios de División

1. **ID3 (Iterative Dichotomiser 3):**

- Solo admite atributos categóricos y no realiza poda.
- Utiliza la **Entropía** $H(S)$ y la **Ganancia de Información** $IG(S, A)$:

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

$$IG(S, A) = H(S) - \sum_{v \in \text{Valores}(A)} \frac{|S_v|}{|S|} H(S_v)$$

2. **C4.5:**
   - Soporta variables continuas mediante la evaluación de umbrales binarios de corte t.
   - Sustituye el IG por el **Gain Ratio** para corregir el sesgo de ID3 hacia atributos con excesivos valores únicos:

$$\text{GainRatio}(S, A) = \frac{IG(S, A)}{\text{SplitInfo}(A)}$$

$$\text{SplitInfo}(A) = -\sum_{i=1}^{k} \frac{|S_i|}{|S|} \log_2 \left(\frac{|S_i|}{|S|} \right)$$

   - Aplica poda pesimista posterior al entrenamiento.
3. **CART (Classification and Regression Trees):**
   - Realiza exclusivamente divisiones binarias.
   - Minimiza la **Impureza de Gini** en clasificación:

$$G(S) = 1 - \sum_{i=1}^{c} p_i^2$$

   - En regresión, minimiza la varianza/suma de errores cuadráticos (SSE).

1.1.5 5. Clustering (Agrupamiento No Supervisado)

1.1.5.1 5.1 k-means

- **Objetivo:** Particionar X en c clusters disjuntos $\omega_1, \ldots, \omega_c$ minimizando la varianza dentro de cada cluster (suma de distancias al cuadrado al centroide μ_i).
- **Paso a Paso del Algoritmo:**
  1. Inicialización aleatoria de k centroides.
  2. Asignación de cada punto al centroide más cercano.
  3. Reordenamiento y recalculo del punto medio/centroide de cada cluster.
  4. Repetición iterativa hasta convergencia.
- **Selección de k:** Se evalúa empíricamente mediante el **Elbow Method** y métricas como el **Silhouette Score** o el **Davies–Bouldin Score**.

1.1.5.2 5.2 Clustering Jerárquico Aglomerativo (Agglomerative)

- **Mecanismo:** Construye una estructura en árbol (*Dendrograma*) fusionando sucesivamente los dos clusters más cercanos desde un estado inicial donde cada punto es un cluster individual.
- **Criterios de Enlace (Linkage Criteria):**
  - **Single Linkage:** Distancia mínima entre miembros de dos clusters ($d_{\min}(C_1, C_2)$).
  - **Complete Linkage:** Distancia máxima entre miembros de dos clusters ($d_{\max}(C_1, C_2)$).
  - **Average Linkage:** Distancia promedio entre todos los pares de elementos entre clusters.

1.1.6 6. Práctica y Aplicación en Código

```
import numpy as np
import torch
import torch.nn as nn
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# ------------------------------------------------------------------------
# 1. Modelos Clasicos en Scikit-Learn (kNN, Naive Bayes, Decision Trees)
# ------------------------------------------------------------------------
def train_classical_models(X_train, y_train, X_test):
# kNN: requiere estandarizacion previa de caracteristicas
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train_scaled, y_train)

# Gaussian Naive Bayes
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# Arbol de Decision CART (Criterio Gini)
tree = DecisionTreeClassifier(criterion='gini', max_depth=5)
tree.fit(X_train, y_train)

return knn, gnb, tree

# ------------------------------------------------------------------------
# 2. Clustering (k-means)
# ------------------------------------------------------------------------
def run_kmeans_clustering(X_data, n_clusters=3):
kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X_data)
centroids = kmeans.cluster_centers_
return cluster_labels, centroids

# ------------------------------------------------------------------------
# 3. MLP Baseline con Vectorizacion (Aplanado) en PyTorch
```

```
    # ------------------------------------------------------------------------
    class MLPClassifier(nn.Module):
    def __init__(self, input_shape=(3, 64, 64), num_classes=10, hidden_units=[512,
    256], dropout_rate=0.3):
    super(MLPClassifier, self).__init__()
    self.input_dim = input_shape[0] * input_shape[1] * input_shape[2]

    layers = []
    in_dim = self.input_dim
    for h_dim in hidden_units:
    layers.append(nn.Linear(in_dim, h_dim))
    layers.append(nn.ReLU())
    layers.append(nn.Dropout(p=dropout_rate))
    in_dim = h_dim

    layers.append(nn.Linear(in_dim, num_classes))
    self.network = nn.Sequential(*layers)

    def forward(self, x):
    x = x.view(x.size(0), -1)
    return self.network(x)
```

2 Ensembles y descomposición multi-clase

2.1 Apuntes del Tema 2: Ensemble Models & Multiclass Decomposition

2.1.1 1. Motivación y Métodos de Ensamble (*Ensembles*)

2.1.1.1 1.1 Filosofía del Ensamble

- **Limitaciones de un Modelo Individual:**
  - Un modelo demasiado simple puede subajustar (*underfit*), mostrando un alto sesgo (*bias*) y bajo rendimiento.
  - Un modelo excesivamente complejo puede sobreajustar (*overfit*), memorizando el ruido de los datos de entrenamiento y mostrando alta varianza (*variance*).
  - Algunos modelos (como los árboles de decisión no podados) son inestables: pequeñas variaciones en los datos de entrenamiento generan cambios drásticos en la estructura del modelo.
- **Idea Clave:** Combinar múltiples estimadores débiles (*weak learners*) para construir un estimador fuerte (*strong learner*) cuya predicción colectiva sea más precisa, estable y robusta que cualquiera de los modelos individuales por separado.
- **Diversidad:** Para que un ensamble funcione eficazmente, los estimadores individuales deben cometer errores de manera independiente o desacoplada (diversidad de hipótesis).

2.1.2 2. Bagging y Random Forest

2.1.2.1 2.1 Bagging (*Bootstrap Aggregating*)

- **Objetivo:** Reducir la **varianza** de modelos inestables y con alta varianza (ej. árboles de decisión profundos) sin incrementar su sesgo.
- **Mecanismo Paso a Paso:**
  1. **Muestreo Bootstrap:** Dado un dataset $\mathcal{D}$ de N muestras, se generan B subconjuntos $\mathcal{D}_b$ extrayendo N muestras con reemplazo de $\mathcal{D}$.
  2. **Entrenamiento Independiente:** Se entrena un estimador $h_b(x)$ de forma completamente independiente sobre cada subconjunto $\mathcal{D}_b$.
  3. **Agregación:**

- **Clasificación:** Voto mayoritario:

$$\hat{y} = \arg\max_{c} \sum_{b=1}^{B} \mathbb{I}(h_b(x) = c)$$

- **Regresión:** Promedio simple:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} h_b(x)$$

2.1.2.2 2.2 Random Forest

- **Mejora sobre Bagging:** En el Bagging estándar, si existe una característica dominante, todos los árboles la elegirán en la raíz, resultando en un conjunto de árboles altamente correlacionados.
- **Decorrelación de Árboles:** Random Forest añade una segunda fuente de aleatoriedad: en cada división (*split*) de cada árbol, se selecciona al azar un subconjunto reducido de m características de las d totales disponibles ($m \ll d$).
  - Para clasificación: típicamente $m = \sqrt{d}$.
  - Para regresión: típicamente $m = \frac{d}{3}$.
- **Evaluación Out-Of-Bag (OOB):** Dado que cada muestra *bootstrap* deja fuera aproximadamente un 36.8% de los datos ($1 - 1/e \approx 0.632$ incluidos), estas muestras "no vistas" sirven como conjunto de validación implícito (*OOB score*) sin requerir un split de validación adicional.

2.1.3 3. Boosting: AdaBoost y Gradient Boosting (XGBoost)

2.1.3.1 3.1 AdaBoost (*Adaptive Boosting*)

- **Mecanismo:** Entrena estimadores de forma **secuencial**. Mantiene una distribución de pesos $D_t(i)$ sobre las muestras del dataset.
- **Algoritmo Paso a Paso:**
  1. Inicializa los pesos de las muestras uniformemente: $D_1(i) = 1/N$.
  2. Para cada iteración $t = 1, \ldots, T$:
     - Entrena un estimador débil $h_t(x)$ minimizando el error ponderado:

$$\epsilon_t = \sum_{i: y_i \neq h_t(x_i)} D_t(i)$$

     - Calcula la importancia/confianza del estimador en la decisión final:

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \epsilon_t}{\epsilon_t}\right)$$

     - Actualiza los pesos de las muestras incrementando la importancia de las mal clasificadas:

$$D_{t+1}(i) \propto D_t(i) \cdot \exp\left(-\alpha_t y_i h_t(x_i)\right)$$

     - Renormaliza D_{t+1} para que forme una distribución válida que sume 1.

3. **Predicción Final:** Voto ponderado:

$$\hat{y} = \text{signo}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

2.1.3.2 3.2 Gradient Boosting y XGBoost

- **Concepto Clave:** En lugar de reponderar muestras, cada nuevo árbol $h_t(x)$ se ajusta directamente para predecir los **residuos** (o el gradiente negativo de la función de pérdida) del ensamble acumulado $F_{t-1}(x)$.
- **Ajuste Aditivo:**

$$F_t(x) = F_{t-1}(x) + \nu h_t(x)$$

  donde $\nu \in (0, 1]$ es el parámetro de tasa de aprendizaje (*shrinkage*) que regula la contribución de cada nuevo árbol para evitar el sobreajuste.
- **Residuos para Error Cuadrático Medio (MSE):**

$$r_{i,t-1} = -\left[\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{t-1}} = y_i - F_{t-1}(x_i)$$

- **XGBoost (*Extreme Gradient Boosting*):** Extensión altamente eficiente que incluye:
  - Expansión de Taylor de segundo orden (utiliza tanto el gradiente de primer orden g_i como el Hessiano h_i).
  - Regularización L_1 (α) y L_2 (λ) sobre las hojas del árbol.
  - Manejo automático e intrínseco de valores faltantes (*sparse-aware split finding*).

2.1.4 4. Descomposición Multiclase: OVA y OVO

Cuando un modelo de clasificación binario puro (ej. Regresión Logística, SVM) debe aplicarse a un problema de K clases, se emplean estrategias de descomposición:

Estrategia	Cantidad de Clasificadores	Configuración del Entrenamiento	Mecanismo de Predicción
One-vs-All (OVA) *(One-vs-Rest)*	K clasificadores	El clasificador k se entrena con la clase k como positiva (+1) y todas las demás como negativas (-1).	Se asigna la clase cuyo clasificador entregue la mayor probabilidad o puntuación (*logit*): $\hat{y} = \arg\max_k f_k(x)$.
One-vs-One (OVO)	$\frac{K(K-1)}{2}$ clasificadores	Se entrena un clasificador para cada par único de clases (c_i, c_j) ignorando el resto.	Cada sub-clasificador emite un voto por su clase preferida. Gana la clase con **mayor número de votos acumulados**.

2.1.5 5. Práctica y Aplicación en Código (Scikit-Learn)

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
```

```
    from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier,
    AdaBoostClassifier
    from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
    from sklearn.svm import SVC
    from sklearn.metrics import accuracy_score

    X, y = make_classification(
    n_samples=1000,
    n_features=20,
    n_informative=12,
    n_classes=4,
    random_state=42
    )

    X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
    )

    def train_and_evaluate_ensembles(X_tr, y_tr, X_te, y_te):
    models = {
        "Random Forest": RandomForestClassifier(
        n_estimators=100, max_features="sqrt", random_state=42
        ),
        "AdaBoost": AdaBoostClassifier(
        n_estimators=50, learning_rate=0.1, random_state=42
        ),
        "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42
        )
    }

    print("Evaluacion de Metodos de Ensamble")
    for name, model in models.items():
    model.fit(X_tr, y_tr)
    preds = model.predict(X_te)
    acc = accuracy_score(y_te, preds)
    print(f"Modelo: {name} | Test Accuracy: {acc:.4f}")

    def train_multiclass_decomposition(X_tr, y_tr, X_te, y_te):
    base_binary_clf = SVC(kernel='linear', C=1.0, random_state=42)

    ova_model = OneVsRestClassifier(base_binary_clf)
    ova_model.fit(X_tr, y_tr)
    ova_preds = ova_model.predict(X_te)

    ovo_model = OneVsOneClassifier(base_binary_clf)
    ovo_model.fit(X_tr, y_tr)
    ovo_preds = ovo_model.predict(X_te)

    print("Estrategias de Descomposicion Multiclase")
    print(f"Numero de estimadores OVA (K=4): {len(ova_model.estimators_)}")
    print(f"Accuracy OVA: {accuracy_score(y_te, ova_preds):.4f}")

    print(f"Numero de estimadores OVO: {len(ovo_model.estimators_)}")
    print(f"Accuracy OVO: {accuracy_score(y_te, ovo_preds):.4f}")

    if __name__ == "__main__":
    train_and_evaluate_ensembles(X_train, y_train, X_test, y_test)
```

```
61     train_multiclass_decomposition(X_train, y_train, X_test, y_test)
```

3 Regresión lineal

3.1 Apuntes del Tema 3: Regresión Lineal

3.1.1 1. Introducción y Planteamiento del Modelo

- **Definición:** Modelo paramétrico supervisado para predecir un valor objetivo continuo $y \in \mathbb{R}$ a partir de un vector de características $x \in \mathbb{R}^d$.
- **Ecuación del Modelo:** $\hat{y} = x^\top w + b$.

3.1.2 2. Función de Pérdida (MSE)

- **MSE:** $\mathcal{L}(w, b) = \frac{1}{n}\|y - (Xw + b\mathbf{1})\|_2^2$.
- **Derivación Probabilística:** Surge al asumir ruido gaussiano $\epsilon \sim \mathcal{N}(0, \sigma^2)$, donde maximizar la verosimilitud equivale a minimizar el MSE.

3.1.3 3. Optimización

- **Analítica:** $\tilde{w} = (\tilde{X}^\top \tilde{X})^{-1}\tilde{X}^\top y$. (Costoso $O(d^3)$).
- **Descenso de Gradiente:** Actualización iterativa basada en el gradiente del MSE.

3.1.4 4. Escalado y Expansión Polinomial

- **Estandarización:** Fundamental para evitar el "zig-zag" en el descenso de gradiente.
- **Expansión Polinomial:** Permite capturar fronteras no lineales conservando la linealidad en parámetros, aumentando el riesgo de *overfitting*.

3.2 Conexión con la práctica

3.2.1 Fuente: `03_01_Linear_regression.ipynb`

```
    # Implementacion basica con PyTorch
    x = np.random.uniform(-3, 3, 200)
    y = w_true*x + b_true + noise
```

4 Regresión logística

4.1 Apuntes del Tema 4: Regresión Logística

4.1.1 1. Concepto y Función Sigmoide

- **Objetivo:** Clasificación binaria prediciendo $P(y = 1|x) \in (0, 1)$.
- **Sigmoide:** $\sigma(z) = \frac{1}{1+e^{-z}}$, donde $z = x^\top w + b$.

4.1.2 2. Función de Pérdida (BCE)

- **Binary Cross-Entropy:** $\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$.
- **Propiedad:** Estrictamente convexa, garantiza convergencia al mínimo global.
- **Derivación Probabilística:** Equivalente a la Máxima Verosimilitud (MLE) bajo una distribución de Bernoulli.

4.1.3 3. Regularización (L_1 y L_2)

- **Ridge (L_2):** Penaliza w^2 (esfera), reduce magnitud de pesos.
- **Lasso (L_1):** Penaliza $|w|$ (diamante), induce dispersión (*sparsity*), lleva pesos a cero.

4.1.4 4. Práctica

```
    # Pipeline de Scikit-Learn con escalado, poly y logistic
    pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2)),
    ('logistic', LogisticRegression(penalty='l2'))
    ])
```

4.2 Autoevaluación

- **P:** ¿Por qué la salida logística es una probabilidad? **R:** La sigmoide mapea el espacio real al intervalo $(0, 1)$.
- **P:** ¿Riesgo de polinomio grado alto? **R:** Sobreajuste; requiere regularización.

5 Perceptrones multicapa

5.1 Apuntes del Tema 5: Multilayer Perceptron (MLP I y II)

5.1.1 1. De Modelos Lineales a No Lineales

5.1.1.1 1.1 Limitaciones de la Transformación Afín

- **Modelos Lineales:** Modelos como la Regresión Lineal o la Regresión Logística calculan transformaciones afines de la forma $z = x^\top w + b$.
- **El Problema del Colapso Lineal:** Si se apilan múltiples capas compuestas únicamente por transformaciones afines, la red colapsa matemáticamente en un único modelo lineal equivalente.
- *Demostración:* Dadas dos capas lineales $H = XW^{(1)} + b^{(1)}$ y $O = HW^{(2)} + b^{(2)}$:

  $$O = (XW^{(1)} + b^{(1)})W^{(2)} + b^{(2)} = X(W^{(1)}W^{(2)}) + (b^{(1)}W^{(2)} + b^{(2)}) = XW + b$$

  donde $W = W^{(1)}W^{(2)}$ y $b = b^{(1)}W^{(2)} + b^{(2)}$. La capacidad expresiva no aumenta con respecto a un modelo de una sola capa.
- **Separabilidad No Lineal (Problema XOR):** Las fronteras de decisión rectilíneas son incapaces de separar clases complejas o disjuntas (como la función lógica XOR o patrones circulares/concéntricos).

5.1.2 2. Capas Ocultas y Funciones de Activación

Para dotar al Perceptrón Multicapa (MLP) de la capacidad de aprender relaciones no lineales, se aplica una **función de activación no lineal** $\phi(\cdot)$ elemento a elemento tras cada transformación afín en las capas ocultas.

```
Entrada X (n x d) ---> [ Z = X.W(1) + b(1) ] ---> [ H = phi(Z) ] ---> Salida O
```

5.1.2.1 2.1 Principales Funciones de Activación

5.1.3 3. Clasificación Multiclase: Regresión Softmax

Cuando el target y pertenece a una de las K categorías posibles ($K > 2$) sin un orden explícito, se emplea la codificación **One-Hot Encoding** ($y \in \mathbb{R}^K$).

Función	Ecuación $\phi(z)$	Rango	Propiedades y Derivada $\phi'(z)$
Sigmoide	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Interpretación probabilística. **Problema:** Zonas llanas donde $\sigma'(z) \approx 0$, provocando desvanecimiento de gradiente (*vanishing gradient*).
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Centrada en cero. Sufre de desvanecimiento para $\|z\|$ elevados ($\tanh'(z) = 1 - \tanh^2(z)$).
ReLU	$\text{ReLU}(z) = \max(0, z)$	$[0, \infty)$	**Estándar en redes profundas:** Eficiente. Derivada $\phi'(z) = 1$ para $z > 0$. **Riesgo:** *Dying ReLU* si $z \leq 0$ permanentemente.

5.1.3.1 3.1 Operador Softmax

La capa de salida genera K valores no acotados denominados **logits** ($O = [o_1, o_2, \ldots, o_K]$). Para transformar los logits en una distribución de probabilidad bien definida donde $\sum_{k=1}^{K} \hat{y}_k = 1$ y $\hat{y}_k > 0$, se aplica la función **Softmax**:

$$\hat{y}_k = \text{Softmax}(o_k) = \frac{e^{o_k}}{\sum_{j=1}^{K} e^{o_j}}$$

- **Propiedad Conservadora del Orden:** Preserva la ordenación monótona de los logits ($\arg\max_k \hat{y}_k = \arg\max_k o_k$).

5.1.3.2 3.2 Pérdida de Entropía Cruzada Categorical (*Categorical Cross-Entropy*)

Para una muestra con vector One-Hot y y predicciones $\hat{y}$:

$$\mathcal{L}(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

> **Nota de Implementación (PyTorch):** En la práctica, no se añade manualmente la capa `Softmax` al final del modelo. Se retorna el tensor de logits directamente y se utiliza `nn.CrossEntropyLoss()`, la cual combina internamente `Softmax` y `Cross-Entropy` de forma numéricamente estable mediante el truco *log-sum-exp*.

5.1.4 4. Algoritmo de Propagación: Forward Pass y Backpropagation

5.1.4.1 4.1 Forward Pass (Propagación hacia Adelante)

Para un MLP de L capas con entrada $X \in \mathbb{R}^{n \times d}$ ($H^{(0)} = X$): Para $i = 1, 2, \ldots, L-1$:

$$Z^{(i)} = H^{(i-1)} W^{(i)} + b^{(i)}$$

$$H^{(i)} = \phi(Z^{(i)})$$

Salida final (L):

$$O = H^{(L-1)} W^{(L)} + b^{(L)}$$

5.1.4.2 4.2 Backpropagation (Propagación hacia Atrás)

Consiste en la aplicación sistemática de la **Regla de la Cadena** multivariante desde la función de pérdida $\mathcal{J}$ hasta las primeras capas para obtener los gradientes $\frac{\partial \mathcal{J}}{\partial W^{(i)}}$ y $\frac{\partial \mathcal{J}}{\partial b^{(i)}}$:

$$\frac{\partial \mathcal{J}}{\partial W^{(i)}} = (H^{(i-1)})^\top \cdot \delta^{(i)}, \quad \text{donde } \delta^{(i)} = \frac{\partial \mathcal{J}}{\partial Z^{(i)}}$$

- **Gestión de Memoria:** Los tensores intermedios calculados durante el *forward pass* $(H^{(i)}, Z^{(i)})$ deben almacenarse en memoria para ser reevaluados durante el *backpropagation*, por lo que entrenar consume considerablemente más memoria que ejecutar inferencia (`with torch.no_grad():`).

5.1.5 5. Propiedades Teóricas e Inicialización de Parámetros

5.1.5.1 5.1 Teorema de Aproximación Universal

- **Definición:** Un Perceptrón Multicapa con tan solo **una capa oculta** y un número suficiente de neuronas no lineales puede aproximar cualquier función continua en un dominio compacto con precisión arbitraria $\epsilon > 0$.
- **¿Por qué usar redes profundas?** El teorema demuestra la *existencia* teórica, pero no especifica la cantidad de neuronas requeridas (que puede ser masiva/exponencial) ni si se pueden aprender eficazmente. Añadir profundidad permite representar funciones complejas mediante composiciones jerárquicas con un **número drásticamente menor de parámetros**.

5.1.5.2 5.2 Rompimiento de la Simetría e Inicialización

- **Inconveniente de Inicialización en Cero:** Si todos los pesos de una capa oculta se inicializan con la misma constante (p. ej. $w_{ij} = 0$), todas las neuronas calcularán el mismo valor en el *forward* y recibirán el mismo gradiente en el *backward*. El MLP colapsaría al comportamiento de un modelo lineal único.
- **Estrategias de Inicialización:**
  - **Xavier / Glorot Initialization:** Diseñada para activaciones sigmoideas o tanh. Mantiene constante la varianza a través de las capas:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

  - **He / Kaiming Initialization:** Diseñada para activaciones **ReLU**, compensando que la mitad de las neuronas se desactivan (≤ 0):

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

5.1.6 6. Problemas en Redes Profundas: Desvanecimiento y Explosión del Gradiente

Al encadenar múltiples capas mediante la regla de la cadena, la magnitud del gradiente acumulado en las capas iniciales se calcula como un producto iterativo:

$$\frac{\partial \mathcal{J}}{\partial W^{(1)}} \propto \prod_{l=2}^{L} W^{(l)} \cdot \phi'(Z^{(l)})$$

- **Vanishing Gradients (Desvanecimiento):** Si las derivadas o valores propios son menores a 1 (p. ej., sigmoide/tanh en saturación), el producto decrece exponencialmente hacia 0. Las capas iniciales dejan de aprender.
- **Exploding Gradients (Explosión):** Si los valores de los pesos son mayores a 1, el producto crece exponencialmente provocando inestabilidad numérica (`NaN` o `Inf`). Se mitiga aplicando **Gradient Clipping** (`nn.utils.clip_grad_norm_`).

5.1.7 7. Práctica y Aplicación en Código (PyTorch)

```
import torch
import torch.nn as nn
import torch.optim as optim

class DeepMLP(nn.Module):
def __init__(self, input_dim=8, hidden_dims=[12, 6], num_classes=3, dropout_rate
=0.2):
super(DeepMLP, self).__init__()

layers = []
in_dim = input_dim

for h_dim in hidden_dims:
linear_layer = nn.Linear(in_dim, h_dim)
nn.init.kaiming_normal_(linear_layer.weight, nonlinearity='relu')
nn.init.zeros_(linear_layer.bias)

layers.append(linear_layer)
layers.append(nn.ReLU())
layers.append(nn.Dropout(p=dropout_rate))
in_dim = h_dim

out_layer = nn.Linear(in_dim, num_classes)
nn.init.xavier_normal_(out_layer.weight)
nn.init.zeros_(out_layer.bias)
layers.append(out_layer)

self.network = nn.Sequential(*layers)

def forward(self, x):
return self.network(x)

def train_mlp_multiclass():
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

X_dummy = torch.randn(100, 8)
y_dummy = torch.randint(0, 3, (100,))

model = DeepMLP(input_dim=8, hidden_dims=[12, 6], num_classes=3).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.005)

X_b = X_dummy.to(device)
y_b = y_dummy.to(device)

model.train()
```

```
    for epoch in range(1, 101):
    optimizer.zero_grad()
    logits = model(X_b)
    loss = criterion(logits, y_b)
    loss.backward()

    nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()

    if epoch % 25 == 0:
    print(f"Epoca [{epoch:3d}/100] | Loss: {loss.item():.4f}")

    if __name__ == "__main__":
    train_mlp_multiclass()
```

5.2 Conexión con las prácticas

Las prácticas muestran el paso de regresión logística multiclase a MLP y relacionan `nn.Linear`, `nn.Sequential` y `autograd` con el forward/backward pass.

5.3 Autoevaluación razonada

Pregunta

¿Qué representa un logit en clasificación multiclase? **Solución.** La puntuación sin normalizar de una clase antes de aplicar la transformación usada por la pérdida o la probabilidad.

Pregunta

¿Qué calcula autograd? **Solución.** Construye y recorre el grafo de operaciones para obtener derivadas respecto de los tensores que requieren gradiente.

6 Regularización en redes profundas

6.1 Apuntes del Tema 6: Regularización en Redes Profundas

6.1.1 1. El Problema del Sobreajuste (*Overfitting*) en Redes Profundas

6.1.1.1 1.1 Naturaleza del Problema

- **Alta Capacidad Expresiva:** Las redes neuronales profundas poseen millones de parámetros libres. Esta gran capacidad les permite memorizar no solo los patrones subyacentes de los datos, sino también el ruido de entrenamiento.
- **Brecha de Generalización (*Generalization Gap*):** Se define como la diferencia entre la pérdida/error en el conjunto de validación (o prueba) y el conjunto de entrenamiento.
  - *Baja varianza / Buen ajuste:* La pérdida de validación disminuye a la par con la de entrenamiento.
  - *Alta varianza / Overfitting:* A partir de cierto punto, la pérdida de entrenamiento sigue bajando mientras la pérdida de validación empieza a crecer, ensanchando la brecha de generalización.
- **Objetivo de la Regularización:** Modificar o restringir el proceso de aprendizaje para reducir el error de prueba/generalización sin incrementar excesivamente el error de entrenamiento.

6.1.2 2. Decaimiento de Pesos (*Weight Decay* / Regularización L_2 vs. L_1)

La forma clásica de regularización consiste en añadir un término de penalización $\Omega(W)$ a la función de pérdida original $\mathcal{L}(W)$:

$$\mathcal{J}(W) = \mathcal{L}(W) + \lambda\Omega(W)$$

donde $\lambda > 0$ es el hiperparámetro que controla la intensidad de la regularización.

6.1.2.1 2.1 Decaimiento de Pesos (L_2 Regularization)

- **Penalización:** Se utiliza la norma L_2 al cuadrado de las matrices de pesos:

$$\Omega(W) = \frac{1}{2}\|W\|_2^2 = \frac{1}{2}\sum_i \sum_j w_{ij}^2$$

- **Efecto en el Gradiente y Actualización (SGD):**

$$\frac{\partial \mathcal{J}}{\partial W} = \frac{\partial \mathcal{L}}{\partial W} + \lambda W$$

$$W \leftarrow W - \eta \frac{\partial \mathcal{J}}{\partial W} = W(1 - \eta\lambda) - \eta \frac{\partial \mathcal{L}}{\partial W}$$

- **Interpretación:** Antes de aplicar el paso de gradiente habitual, los pesos se reducen por un factor multiplicativo $(1 - \eta\lambda) < 1$. Esto fuerza a la red a mantener pesos pequeños y distribuidos de forma suave, previniendo comportamientos extremos frente a pequeñas variaciones en la entrada.

6.1.2.2 2.2 Regularización L_1 (Lasso)

- **Penalización:** Suma de los valores absolutos de los pesos:

$$\Omega(W) = \|W\|_1 = \sum_i \sum_j |w_{ij}|$$

- **Efecto:** Mantiene una magnitud de penalización constante $\lambda \cdot \text{signo}(W)$ independientemente del tamaño del peso, lo que empuja muchos pesos **exactamente a cero**, generando matrices de pesos esparsas (*sparse models*) y actuando como selección implícita de características.

6.1.3 3. Dropout (Inyección de Ruido en Activaciones)

Propuesto por Srivastava et al. (2014), es una de las técnicas de regularización más efectivas en redes profundas.

6.1.3.1 3.1 Mecanismo de Funcionamiento

Durante el entrenamiento, en cada iteración o minilote, cada neurona oculta se "apaga" (se fuerza a cero) con una probabilidad $p \in (0, 1)$ elegida de forma independiente.

- **Formulación Matemática (Inverted Dropout):** Dada una activación intermedia h, se transforma en h' mediante:

$$h' = \begin{cases} 0 & \text{con probabilidad } p \\ \frac{h}{1-p} & \text{con probabilidad } 1 - p \end{cases}$$

- **Reescalado ($\frac{1}{1-p}$):** Escalar por $\frac{1}{1-p}$ durante el entrenamiento garantiza que el valor esperado de la activación sea constante ($E[h'] = h$), evitando modificar la magnitud de las entradas en la fase de test.

6.1.3.2 3.2 ¿Por qué funciona Dropout?

1. **Prevención de la Co-adaptación:** Fuerza a cada neurona a aprender características útiles por sí misma, en lugar de depender de la presencia corregidora de otras neuronas específicas.
2. **Ensamble Implícito (*Implicit Ensembling*):** Entrenar con Dropout equivale a entrenar en cada paso un subconjunto distinto de la red (*thinned network*) que comparte pesos. La evaluación final sin Dropout equivale geométricamente a promediar las predicciones de un número masivo de subredes.
3. **Inferencia / Test:** Durante la fase de evaluación, el Dropout debe **desactivarse** llamando a `model.eval()` en PyTorch.

6.1.4 4. Parada Temprana (*Early Stopping*)

Es una técnica de regularización no paramétrica que aprovecha la dinámica temporal del entrenamiento.

6.1.4.1 4.1 Criterio de Funcionamiento

- **Fundamento:** A medida que aumentan las épocas de entrenamiento, el error de entrenamiento disminuye de manera monótona, pero el error de validación alcanza un mínimo y luego empieza a aumentar.
- **Mecanismo:**
  1. Se evalúa el error sobre el conjunto de validación al final de cada época.
  2. Se guarda una copia de los mejores pesos del modelo cuando la pérdida de validación alcanza un nuevo mínimo.
  3. **Criterio de Paciencia (*Patience*):** Si la pérdida de validación no mejora durante un número P determinado de épocas consecutivas, el entrenamiento se interrumpe y se restauran los pesos óptimos guardados.

6.1.5 5. Práctica y Aplicación en Código (PyTorch)

A continuación se presenta la implementación modular que integra **Weight Decay** (L_2), **Dropout** y **Early Stopping** utilizando PyTorch:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

class RegularizedMLP(nn.Module):
def __init__(self, input_dim=20, hidden_dim=64, num_classes=2, dropout_p=0.5):
super(RegularizedMLP, self).__init__()

self.network = nn.Sequential(
nn.Linear(input_dim, hidden_dim),
nn.ReLU(),
nn.Dropout(p=dropout_p),

nn.Linear(hidden_dim, hidden_dim),
nn.ReLU(),
nn.Dropout(p=dropout_p),

nn.Linear(hidden_dim, num_classes)
)

def forward(self, x):
return self.network(x)

class EarlyStopping:
def __init__(self, patience=5, delta=0.001):
self.patience = patience
self.delta = delta
self.best_loss = float('inf')
self.counter = 0
```

```
    self.early_stop = False
    self.best_state_dict = None

    def __call__(self, val_loss, model):
    if val_loss < self.best_loss - self.delta:
    self.best_loss = val_loss
    self.counter = 0
    self.best_state_dict = model.state_dict().copy()
    else:
    self.counter += 1
    if self.counter >= self.patience:
    self.early_stop = True

    def train_with_regularization():
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

    X_train = torch.randn(800, 20)
    y_train = torch.randint(0, 2, (800,))
    X_val = torch.randn(200, 20)
    y_val = torch.randint(0, 2, (200,))

    train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32, shuffle
    =True)
    val_loader = DataLoader(TensorDataset(X_val, y_val), batch_size=32, shuffle=False
    )

    model = RegularizedMLP(input_dim=20, dropout_p=0.3).to(device)
    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
    early_stopper = EarlyStopping(patience=10)

    epochs = 100
    for epoch in range(1, epochs + 1):
    model.train()
    train_loss = 0.0
    for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)
    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()
    train_loss += loss.item() * images.size(0)

    train_loss /= len(train_loader.dataset)

    model.eval()
    val_loss = 0.0
    with torch.no_grad():
    for images, labels in val_loader:
    images, labels = images.to(device), labels.to(device)
    outputs = model(images)
    loss = criterion(outputs, labels)
    val_loss += loss.item() * images.size(0)

    val_loss /= len(val_loader.dataset)

```

```
87      print(f"Epoca [{epoch:3d}/{epochs}] | Train Loss: {train_loss:.4f} | Val Loss: {
        val_loss:.4f}")
88
89      early_stopper(val_loss, model)
90      if early_stopper.early_stop:
91      print(f"\n[Early Stopping] Entrenamiento detenido en la epoca {epoch}.")
92      model.load_state_dict(early_stopper.best_state_dict)
93      break
94
95      if __name__ == '__main__':
96      train_with_regularization()
```

6.2 Conexión con las prácticas

La práctica permite relacionar las curvas de entrenamiento/validación con weight decay, dropout y early stopping.

6.3 Autoevaluación razonada

Pregunta

¿Qué síntoma sugiere sobreajuste? **Solución.** La pérdida de entrenamiento sigue mejorando mientras la validación empeora o deja de mejorar.

Pregunta

¿Qué hace early stopping? **Solución.** Detiene el entrenamiento cuando la métrica de validación no mejora según un criterio y puede restaurar el mejor estado.

7 Descenso de gradiente y optimizadores

7.1 Apuntes del Tema 8: Descenso de Gradiente, Métodos Acelerados y Adaptativos

7.1.1 1. Fundamentos de Optimización en Deep Learning

7.1.1.1 1.1 Optimización Matemática vs. Aprendizaje Profundo

- **Optimización Pura:** Su objetivo principal es minimizar una función objetivo $\mathcal{L}(w)$ sobre un conjunto de datos de entrenamiento para reducir el error de entrenamiento.
- **Deep Learning:** Su verdadero objetivo es reducir el **error de generalización** (desempeño en datos no vistos del conjunto de prueba), combinando la minimización del error con técnicas para mitigar el sobreajuste (*overfitting*).

7.1.1.2 1.2 Retos en la Superficie de Pérdida

1. **Mínimos Locales:** Puntos donde el gradiente es cero ($\nabla f(x) = 0$) y la matriz Hessiana es definida positiva. En redes profundas con alta dimensionalidad, la mayoría de los puntos críticos no son mínimos locales, sino puntos de silla.
2. **Puntos de Silla (*Saddle Points*):** Puntos donde el gradiente se anula, pero la matriz Hessiana posee autovalores tanto positivos como negativos (la función sube en algunas direcciones y baja en otras).
3. **Desvanecimiento del Gradiente (*Vanishing Gradients*):** Regiones extremadamente llanas de la superficie de pérdida (por ejemplo, en las zonas de saturación de $\tanh(x)$ o la sigmoide) donde $\|\nabla f(x)\| \approx 0$, ralentizando drásticamente la actualización de los parámetros.

7.1.1.3 1.3 Convexidad y Series de Taylor

- **Convexidad:** Un conjunto $\mathcal{X}$ es convexo si el segmento que une cualquier par de puntos $a, b \in \mathcal{X}$ está totalmente contenido en $\mathcal{X}$. Una función f es convexa si:

$$\lambda f(x) + (1-\lambda) f(x') \geq f(\lambda x + (1-\lambda)x'), \quad \forall \lambda \in [0,1]$$

  Propiedad clave: En funciones estrictamente convexas, todo mínimo local es también un **mínimo global**. Aunque las funciones de pérdida en redes profundas son no convexas, exhiben comportamiento convexo en las vecindades inmediatas de los mínimos locales.

- **Expansión en Serie de Taylor:** Aproximación polinomial local de una función diferenciable $f(x)$ alrededor de un punto x_0:

$$f(x) \approx f(x_0) + \nabla f(x_0)^\top (x - x_0) + \frac{1}{2}(x - x_0)^\top \mathbf{H}(x - x_0)$$

  donde $\mathbf{H}$ es la matriz Hessiana de segundas derivadas parciales ($\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$).

7.1.2 2. Algoritmos de Descenso de Gradiente

7.1.2.1 2.1 Descenso de Gradiente Unidimensional y Multivariante

Partiendo de la expansión de Taylor de primer orden $f(x+\epsilon) \approx f(x) + \epsilon \nabla f(x)$, para garantizar la máxima disminución del valor de f, debemos movernos en la dirección opuesta al gradiente ($\epsilon = -\eta \nabla f(x)$ con $\eta > 0$):

$$w_{t+1} = w_t - \eta \nabla f(w_t)$$

- **Impacto de la Tasa de Aprendizaje (η):**
  - Si η es demasiado pequeño: Convergencia extremadamente lenta.
  - Si η es demasiado grande: Oscilaciones inestables e incapacidad para converger o divergencia.

7.1.2.2 2.2 Variantes del Descenso de Gradiente según el Tamaño del Lote

Variante	**Tamaño del Lote**	**Ventajas**	**Inconvenientes**
Batch GD	Muestras totales N	Cálculo exacto y convergencia suave.	Inviable en memoria RAM/GPU para grandes datasets; lento.
SGD	1 muestra ($\|B\| = 1$)	Actualizaciones muy rápidas; ayuda a escapar de mínimos locales.	Gradientes con altísima varianza, trayectoria en "zig-zag" muy ruidosa.
Mini-Batch SGD	Lote pequeño ($\|B\| \in [32, 512]$)	**Estándar:** Aprovecha paralelización GPU y suaviza el gradiente.	Requiere afinar la tasa de aprendizaje global η.

7.1.2.3 2.3 Método de Newton (Segundo Orden)

Aprovecha la curvatura local mediante la matriz Hessiana para calcular el paso óptimo:

$$\Delta x = -\mathbf{H}^{-1} \nabla f(x)$$

- **Ventaja:** Ajusta automáticamente la amplitud del paso sin depender fuertemente de η.
- **Desventaja:** Computacionalmente prohibitivo en Deep Learning ($O(d^3)$ para invertir $\mathbf{H}$ con millones de parámetros) y sensible a direcciones donde la Hessiana es definida negativa.

7.1.3 3. Optimizadores Acelerados y Adaptativos

7.1.3.1 3.1 Método del Momento (*Momentum*)

Introduce una variable de velocidad v_t que acumula los gradientes pasados como un promedio móvil ponderado exponencialmente:

$$v_t \leftarrow \beta v_{t-1} + g_t$$

$$w_t \leftarrow w_{t-1} - \eta v_t$$

- **Propiedades:** Amortigua las oscilaciones en direcciones de alta varianza y acelera el avance a lo largo de barrancos. Típicamente $\beta \approx 0.9$.

7.1.3.2 3.2 AdaGrad (*Adaptive Gradient Algorithm*)

Adapta la tasa de aprendizaje de cada parámetro individualmente basándose en la suma acumulada de sus gradientes al cuadrado s_t:

$$s_t \leftarrow s_{t-1} + g_t \odot g_t$$

$$w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

- **Inconveniente:** s_t crece de forma monótona, haciendo que la tasa de aprendizaje efectiva tienda a cero y detenga el aprendizaje prematuramente.

7.1.3.3 3.3 RMSProp y AdaDelta

Resuelven la acumulación desmedida de AdaGrad sustituyendo la suma por un promedio móvil con decaimiento exponencial con parámetro $\gamma \in (0, 1)$:

$$s_t \leftarrow \gamma s_{t-1} + (1 - \gamma) g_t \odot g_t$$

$$w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

7.1.3.4 3.4 Adam (*Adaptive Moment Estimation*)

Combina los beneficios del **Momento** (v_t) y de **RMSProp** (s_t), añadiendo una **corrección de sesgo**:

1. **Primer Momento:** $v_t \leftarrow \beta_1 v_{t-1} + (1 - \beta_1) g_t$
2. **Segundo Momento:** $s_t \leftarrow \beta_2 s_{t-1} + (1 - \beta_2)(g_t \odot g_t)$
3. **Corrección de Sesgo:** $\hat{v}_t = \frac{v_t}{1-\beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1-\beta_2^t}$
4. **Actualización:** $w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{\hat{s}_t + \epsilon}} \odot \hat{v}_t$

Hiperparámetros por defecto estándar: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

7.1.4 4. Políticas de Ajuste (*Learning Rate Schedulers*) y Warmup

7.1.4.1 4.1 Planificadores de Tasa de Aprendizaje (*Schedulers*)

Reducen la tasa de aprendizaje η a medida que avanza el entrenamiento para evitar sobrepasar el mínimo local:

- **Polynomial Decay:** $\eta_t = \eta_0 \left(1 - \frac{t}{T}\right)^p$.
- **Step / Factor Decay:** $\eta_{t+1} = \eta_t \cdot \alpha$ con $\alpha \in (0, 1)$ cada k épocas.
- **Cosine Annealing:** Modula la tasa siguiendo una curva cosenoidal decreciente.
- **ReduceLROnPlateau:** Reduce η por un factor cuando una métrica de validación deja de mejorar.

7.1.4.2 4.2 Calentamiento (*Warmup*)

Inicia con una tasa de aprendizaje muy pequeña durante las primeras épocas e incrementarla linealmente hasta el valor η_0. Previene la desestabilización inicial de los pesos cuando los gradientes son excesivamente volátiles.

7.1.5 5. Práctica y Aplicación en Código (PyTorch)

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import LambdaLR

class DummyNet(nn.Module):
def __init__(self, input_dim=10, num_classes=2):
super(DummyNet, self).__init__()
self.fc = nn.Sequential(
nn.Linear(input_dim, 32),
nn.ReLU(),
nn.Linear(32, num_classes)
)

def forward(self, x):
return self.fc(x)

def setup_optimizers(model):
optimizer_sgd = optim.SGD(
model.parameters(),
lr=0.01,
momentum=0.9,
weight_decay=1e-4
)

optimizer_adam = optim.Adam(
model.parameters(),
lr=0.001,
betas=(0.9, 0.999),
eps=1e-8
)

```

```
    return optimizer_sgd, optimizer_adam

    def get_warmup_scheduler(optimizer, warmup_epochs=5, max_epochs=50):
    def lr_lambda(epoch):
    if epoch < warmup_epochs:
    return float(epoch + 1) / float(warmup_epochs)
    return 0.5 * (1.0 + torch.cos(torch.tensor((epoch - warmup_epochs) / (max_epochs -
     warmup_epochs) * 3.14159)))

    return LambdaLR(optimizer, lr_lambda=lr_lambda)

    def train_demo():
    model = DummyNet()
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
    scheduler = get_warmup_scheduler(optimizer, warmup_epochs=5, max_epochs=20)
    criterion = nn.CrossEntropyLoss()

    inputs = torch.randn(16, 10)
    labels = torch.randint(0, 2, (16,))

    print("Demostraciod de Entrenamiento con Scheduler")
    for epoch in range(1, 21):
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    current_lr = optimizer.param_groups[0]['lr']
    print(f"Epoca [{epoch:2d}/20] | Loss: {loss.item():.4f} | LR: {current_lr:.6f}")

    scheduler.step()

    if __name__ == "__main__":
    train_demo()
```

7.2 Conexión con las prácticas

Las prácticas comparan optimizadores y schedulers en experimentos con datos y pérdidas observables. Las curvas incluidas en las salidas permiten discutir velocidad, estabilidad y sensibilidad al learning rate.

7.3 Autoevaluación razonada

Pregunta

¿Qué problema intenta aliviar momentum? **Solución.** La oscilación y el avance lento en direcciones con curvaturas o escalas muy diferentes.

Pregunta

¿Por qué un scheduler modifica el learning rate? **Solución.** Para adaptar el tamaño de las actualizaciones durante el entrenamiento y favorecer convergencia o refinamiento final.

8 Redes neuronales convolucionales

8.1 Apuntes del Tema 9: Redes Neuronales Convolucionales (CNN I y II)

8.1.1 1. De MLPs a Convoluciones: Motivación e Sesgos Inductivos

8.1.1.1 1.1 Limitaciones del Perceptrón Multicapa (MLP) en Imágenes

- **Explosión de Parámetros:** Un MLP requiere aplanar la imagen de dimensión $C \times H \times W$ a un vector unidimensional $x \in \mathbb{R}^{C \cdot H \cdot W}$.
  - *Ejemplo:* Para una imagen pequeña de $64 \times 64 \times 3 = 12\,288$ entradas, una primera capa oculta con 1 024 neuronas requiere $(12\,288 \times 1\,024) + 1\,024 \approx 12.58$ millones de parámetros.
- **Pérdida de Estructura Espacial:** Al aplanar la matriz de píxeles, el MLP trata la distancia entre píxeles adyacentes de la misma manera que la distancia entre píxeles situados en esquinas opuestas de la imagen.
- **Sensibilidad al Desplazamiento:** Un MLP no reconoce un objeto si se traslada unos pocos píxeles en la imagen, ya que la entrada vectorizada cambia por completo.

8.1.1.2 1.2 Principios de Invariancia Espacial

Para procesar señales estructuradas en 2D de forma eficiente, las Redes Neuronales Convolucionales (CNN) incorporan dos sesgos inductivos clave:

1. **Localidad (*Local Receptive Fields*):** Las características visuales significativas (bordes, esquinas, texturas) se forman por la interacción de píxeles contiguos en regiones espaciales reducidas.
2. **Invariancia a la Traslación / Compartición de Pesos (*Weight Sharing*):** La representación o patrón de un objeto no debe cambiar independientemente de la posición que ocupe dentro de la imagen. Un detector de bordes es útil tanto en la esquina superior izquierda como en el centro.

8.1.2 2. Operaciones Convolucionales y sus Parámetros

8.1.2.1 2.1 La Operación de Convolución 2D

Un filtro o núcleo (*kernel*) $K \in \mathbb{R}^{k_h \times k_w}$ se desliza sobre la matriz de entrada $X \in \mathbb{R}^{H \times W}$, calculando el producto escalar elemento a elemento seguido de una suma:

$$Y_{i,j} = (X * K)_{i,j} = \sum_m \sum_n X_{i+m,j+n} \cdot K_{m,n}$$

8.1.2.2 2.2 Hiperparámetros Fundamentales: Padding, Stride y Canales

- **Relleno (*Padding*, p):** Añade filas y columnas de ceros alrededor del borde de la entrada para controlar la dimensión espacial de salida y evitar la pérdida acelerada de información en los bordes.
- **Paso (*Stride*, s):** Especifica el número de píxeles que se desplaza el filtro en cada paso horizontal y vertical. Un $s > 1$ subsamplea la entrada reduciendo su dimensión espacial.
- **Múltiples Canales de Entrada y Salida ($C_{\text{in}}, C_{\text{out}}$):**
  - Si la entrada tiene C_{in} canales (ej. $RGB = 3$), el filtro posee dimensiones $C_{\text{in}} \times k_h \times k_w$.
  - Para extraer C_{out} mapas de características distintos, se aplican C_{out} filtros independientes de este tipo.

8.1.2.3 2.3 Ecuación de Dimensión de Salida

Dada una entrada de dimensión $C_{\text{in}} \times H \times W$, un kernel de tamaño $k_h \times k_w$, padding p y stride s, la dimensión del mapa de características resultante $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$ se calcula mediante:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k_h}{s} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2p - k_w}{s} \right\rfloor + 1$$

8.1.3 3. Capas de Agregación (*Pooling*)

Las capas de pooling introducen **invariancia local a pequeñas traslaciones y rotaciones**, reduciendo la dimensión espacial de las representaciones y disminuyendo la carga computacional.

8.1.3.1 3.1 Tipos Principales de Pooling

1. **Max Pooling (`nn.MaxPool2d`):** Extrae el valor máximo dentro de la ventana de selección. Resalta las características más prominentes o activaciones más fuertes.
2. **Average Pooling (`nn.AvgPool2d`):** Calcula el promedio de todos los valores de la ventana. Proporciona una representación más suave de la región.

> **Propiedad importante:** Las capas de pooling operan de forma independiente sobre cada canal y **no poseen parámetros entrenables** (W, b).

8.1.4 4. Jerarquía de Representaciones y Arquitecturas Clásicas

8.1.4.1 4.1 Extracción Jerárquica de Características

A medida que la señal avanza a través de capas convolucionales profundas y subsampling:

- **Capas Iniciales:** Filtran patrones simples y locales de bajo nivel (bordes, gradientes de color, orientaciones).
- **Capas Intermedias:** Combinan los bordes para detectar texturas, esquinas y partes de objetos.
- **Capas Profundas:** Ensamblan las partes para representar objetos completos y estructuras semánticas complejas.

8.1.4.2 4.2 Arquitectura Clásica: LeNet-5 (Yann LeCun, 1998)

Fue una de las primeras redes convolucionales con éxito comercial para la clasificación de dígitos manuscritos (MNIST).

- **Estructura Típica:**
  1. **Entrada:** Imagen en escala de grises ($1 \times 28 \times 28$ o $1 \times 32 \times 32$).
  2. **Bloque 1:** `Conv2d` ($1 \to 6$ canales, 5×5) + `Tanh` + `AvgPool2d` (2×2, $s = 2$).
  3. **Bloque 2:** `Conv2d` ($6 \to 16$ canales, 5×5) + `Tanh` + `AvgPool2d` (2×2, $s = 2$).
  4. **Clasificador denso:** Aplanado (`Flatten`) + `Linear` (120) + `Tanh` + `Linear` (84) + `Tanh` + `Linear` (10 clases).

8.1.5 5. Gestión de Datos en PyTorch: `Dataset` y `DataLoader`

Para alimentar eficientemente datos e imágenes a modelos de Deep Learning, PyTorch estandariza la carga mediante dos clases fundamentales:

1. **`torch.utils.data.Dataset`:** Almacena las muestras y sus etiquetas correspondientes. Debe implementar tres métodos:
   - `__init__`: Inicializa rutas, transformaciones y metadatos.
   - `__len__`: Devuelve el número total de muestras en el conjunto de datos.
   - `__getitem__(self, idx)`: Carga y retorna una muestra (X, y) dado un índice idx.
2. **`torch.utils.data.DataLoader`:** Envoltorio que abstrae el procesamiento iterativo en minilotes (*mini-batches*), barajado de datos (*shuffling*), carga paralela multiproceso (*multi-processing*) y gestión de memoria.

8.1.6 6. Práctica y Aplicación en Código (PyTorch y Torchvision)

A continuación se incluye una implementación modular completa que incluye la definición de un `Dataset` personalizado, la canalización de transformaciones, la arquitectura CNN tipo LeNet-5 y el bucle de entrenamiento:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms

IMAGE_SIZE = 32

transform_train = transforms.Compose([
transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
transforms.RandomHorizontalFlip(p=0.5),
transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

transform_val_test = transforms.Compose([
transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

```
    ])

    class LeNet5Classifier(nn.Module):
    def __init__(self, num_classes=10):
    super(LeNet5Classifier, self).__init__()

    self.feature_extractor = nn.Sequential(
    nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=0),
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2),

    nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0),
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2)
    )

    self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120),
    nn.Tanh(),
    nn.Dropout(p=0.3),
    nn.Linear(120, 84),
    nn.Tanh(),
    nn.Linear(84, num_classes)
    )

    def forward(self, x):
    x = self.feature_extractor(x)
    logits = self.classifier(x)
    return logits

    def train_cnn_demo():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    model = LeNet5Classifier(num_classes=10).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)

    dummy_images = torch.randn(16, 3, 32, 32).to(device)
    dummy_labels = torch.randint(0, 10, (16,)).to(device)

    model.train()
    optimizer.zero_grad()

    logits = model(dummy_images)
    loss = criterion(logits, dummy_labels)

    loss.backward()
    optimizer.step()

    print(f"Demostracion exitosa en {device}.")
    print(f"Dimension de Salida (Logits): {logits.shape}")
    print(f"Perdida Inicial (Loss): {loss.item():.4f}")

    if __name__ == "__main__":
    train_cnn_demo()
```

8.2 Conexión con las prácticas

Las prácticas implementan convolución desde cero, kernels aprendidos, `nn.Conv2d`, pooling, LeNet y clasificación de imágenes; las figuras guardadas muestran las representaciones y métricas producidas.

8.3 Autoevaluación razonada

Pregunta

¿Qué controlan padding y stride? **Solución.** Padding controla el borde y el tamaño espacial disponible; stride controla cuánto se desplaza el kernel y, por tanto, la reducción espacial.

Pregunta

¿Por qué se comparten pesos en una CNN? **Solución.** Para detectar el mismo patrón en distintas posiciones con muchos menos parámetros que una capa totalmente conectada.

9 Datos secuenciales y redes recurrentes

9.1 Apuntes del Tema 10: Redes Neuronales Recurrentes

9.1.1 1. De Datos Estáticos a Datos Secuenciales

9.1.1.1 1.1 Limitaciones de los Modelos Feedforward (MLPs y CNNs)

- **Asunción de Muestras Independientes (*i.i.d.*):** Los modelos tradicionales como los Perceptrones Multicapa (MLP) o las Redes Convolucionales (CNN) asumen que las muestras de entrada son independientes e idénticamente distribuidas.
- **Entradas de Tamaño Fijo:** Un MLP o una CNN requieren un vector o tensor de dimensiones de entrada rígidamente predefinidas.
- **Incapacidad para Capturar Dependencias Temporales:** En datos con orden temporal o secuencial (como texto, audio, series temporales financieras o trayectorias GPS), la información del elemento actual depende de los elementos anteriores.

9.1.1.2 1.2 Modelos Autorregresivos y el Estado Oculto

Para modelar secuencias $X = (x_1, x_2, \ldots, x_T)$, deseamos estimar la probabilidad conjunta mediante la regla de la cadena de probabilidades:

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

- **Modelos de Markov (Orden k):** Asumen que x_t solo depende de los k pasos anteriores.
- **Mecanismo de Estado Oculto (*Hidden State*):** En lugar de truncar la historia a k pasos, una Red Neuronal Recurrente (RNN) mantiene un **estado latente** h_t que actúa como una memoria comprimida de toda la secuencia procesada desde $t = 1$ hasta $t - 1$.

9.1.2 2. Arquitectura de la Red Neuronal Recurrente Básica (RNN Vanilla)

9.1.2.1 2.1 Ecuaciones del Estado Oculto y Salida

En cada paso de tiempo t, la RNN recibe la entrada actual $x_t \in \mathbb{R}^d$ y el estado oculto del paso anterior $h_{t-1} \in \mathbb{R}^h$.

1. **Actualización del Estado Oculto:**

$$h_t = \phi\left(x_t W_{xh} + h_{t-1} W_{hh} + b_h\right)$$

2. **Cálculo de la Salida:**

$$\hat{y}_t = \text{Softmax}\left(h_t W_{hq} + b_q\right) \quad \text{o} \quad o_t = h_t W_{hq} + b_q$$

donde:

- $W_{xh} \in \mathbb{R}^{d \times h}$: Matriz de pesos para la entrada.
- $W_{hh} \in \mathbb{R}^{h \times h}$: Matriz de pesos para la transición recurrente de estados.
- $W_{hq} \in \mathbb{R}^{h \times q}$: Matriz de pesos para la capa de salida.
- $\phi(\cdot)$: Función de activación no lineal (típicamente tanh o ReLU).

> **Compartición de Pesos (*Weight Sharing* Temporal):** Las matrices de pesos $\{W_{xh}, W_{hh}, W_{hq}\}$ son exactamente las mismas en todos los pasos de tiempo $t = 1, \ldots, T$.

9.1.3 3. Entrenamiento: Retropropagación a través del Tiempo (BPTT)

9.1.3.1 3.1 Backpropagation Through Time (BPTT)

Para entrenar las RNNs, la red recurrente se "despliega" a lo largo de los T pasos temporales formando un grafo acíclico dirigido. La pérdida total es la suma de las pérdidas en cada paso de tiempo:

$$\mathcal{L}(W) = \sum_{t=1}^{T} \mathcal{L}_t(y_t, \hat{y}_t)$$

Para calcular el gradiente respecto a la matriz de transición recurrente W_{hh}, se aplica la regla de la cadena acumulando las dependencias pasadas:

$$\frac{\partial \mathcal{L}_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial h_t} \cdot \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_k}{\partial W_{hh}}$$

9.1.3.2 3.2 Problema Crítico: Desvanecimiento y Explosión del Gradiente

El término $\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$ implica multiplicar repetidamente la matriz de pesos $W_{hh}^{\top}$.

- **Desvanecimiento del Gradiente (*Vanishing Gradients*):** Si los autovalores de W_{hh} son menores al valor crítico, la magnitud del gradiente decrece exponencialmente hacia cero cuando la secuencia es larga ($T \gg 0$). La red **olvida las dependencias a largo plazo**.
- **Explosión del Gradiente (*Exploding Gradients*):** Si los autovalores son mayores a 1, el gradiente crece exponencialmente destruyendo el entrenamiento. Se soluciona mediante **Gradient Clipping**:

$$g \leftarrow \min\left(1, \frac{\theta}{\|g\|}\right) g$$

9.1.4 4. Redes Gated: LSTM y GRU

Para solucionar el desvanecimiento de gradiente y permitir recordar información a largo plazo, se crearon arquitecturas basadas en puertas de control (*gates*).

9.1.4.1 4.1 LSTM (*Long Short-Term Memory*)

La LSTM sustituye la celda estándar por una estructura de memoria que mantiene un **estado de celda** C_t (*Cell State* o autopista de información) y un **estado oculto** h_t.

Puertas y Ecuaciones Matemáticas de la LSTM:

1. **Forget Gate (f_t):** Decide qué información descartar del estado de celda anterior.
$$f_t = \sigma\left(x_t W_{xf} + h_{t-1} W_{hf} + b_f\right)$$
2. **Input Gate (i_t) y Candidato ($\tilde{C}_t$):** Deciden qué nueva información almacenar.
$$i_t = \sigma\left(x_t W_{xi} + h_{t-1} W_{hi} + b_i\right)$$
$$\tilde{C}_t = \tanh\left(x_t W_{xc} + h_{t-1} W_{hc} + b_c\right)$$
3. **Actualización del Cell State (C_t):**
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$
4. **Output Gate (o_t) y Estado Oculto (h_t):**
$$o_t = \sigma\left(x_t W_{xo} + h_{t-1} W_{ho} + b_o\right)$$
$$h_t = o_t \odot \tanh(C_t)$$

9.1.4.2 4.2 GRU (*Gated Recurrent Unit*)

Variante simplificada de la LSTM que combina el estado de celda y el estado oculto en una sola variable h_t, reduciendo el coste computacional y el número de parámetros.

1. **Reset Gate (r_t):** Determina cuánta información del pasado olvidar:
$$r_t = \sigma\left(x_t W_{xr} + h_{t-1} W_{hr} + b_r\right)$$
2. **Update Gate (z_t):** Actúa simultáneamente como puerta de olvido y entrada:
$$z_t = \sigma\left(x_t W_{xz} + h_{t-1} W_{hz} + b_z\right)$$
3. **Candidato de Estado Oculto ($\tilde{h}_t$):**
$$\tilde{h}_t = \tanh\left(x_t W_{xh} + (r_t \odot h_{t-1}) W_{hh} + b_h\right)$$
4. **Estado Oculto Final (h_t):**
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

9.1.5 5. RNNs Bidireccionales y Apiladas (*Deep RNNs*)

- **RNN Bidireccional (BiRNN / BiLSTM):** Procesa la secuencia en ambas direcciones simultáneamente usando dos estados ocultos independientes ($\overrightarrow{h}_t$ hacia adelante y $\overleftarrow{h}_t$ hacia atrás). El estado combinado es $h_t = [\overrightarrow{h}_t ; \overleftarrow{h}_t]$.
  - *Uso:* Crucial en tareas de procesamiento de lenguaje natural (NLP) donde el contexto futuro influye en el significado del presente.
- **RNN Profunda / Apilada (*Stacked RNN*):** Apila múltiples capas recurrentes donde la salida del estado oculto de la capa l sirve como entrada x_t para la capa $l+1$.

9.1.6 6. Práctica y Aplicación en Código (PyTorch)

A continuación se incluye una implementación modular completa que ilustra la construcción de una red **LSTM Bidireccional** para la clasificación de secuencias (ej. análisis de sentimiento o series temporales) en PyTorch:

```
import torch
import torch.nn as nn
import torch.optim as optim

class SequenceClassifier(nn.Module):
def __init__(self, vocab_size=1000, embed_dim=64, hidden_dim=128, num_classes=2,
num_layers=2, dropout=0.3):
super(SequenceClassifier, self).__init__()

self.embedding = nn.Embedding(vocab_size, embed_dim)

self.lstm = nn.LSTM(
input_size=embed_dim,
hidden_size=hidden_dim,
num_layers=num_layers,
batch_first=True,
bidirectional=True,
dropout=dropout if num_layers > 1 else 0.0
)

self.fc = nn.Linear(hidden_dim * 2, num_classes)
self.dropout = nn.Dropout(p=dropout)

def forward(self, x):
embedded = self.embedding(x)
lstm_out, (h_n, c_n) = self.lstm(embedded)

hidden_forward = h_n[-2, :, :]
hidden_backward = h_n[-1, :, :]
last_hidden = torch.cat((hidden_forward, hidden_backward), dim=1)

out = self.dropout(last_hidden)
logits = self.fc(out)
return logits

def train_rnn_demo():
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

VOCAB_SIZE = 1000
BATCH_SIZE = 16
SEQ_LEN = 32
NUM_CLASSES = 2

model = SequenceClassifier(
vocab_size=VOCAB_SIZE,
embed_dim=64,
hidden_dim=128,
num_classes=NUM_CLASSES,
num_layers=2
).to(device)

criterion = nn.CrossEntropyLoss()
```

```
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    dummy_input = torch.randint(0, VOCAB_SIZE, (BATCH_SIZE, SEQ_LEN)).to(device)
    dummy_labels = torch.randint(0, NUM_CLASSES, (BATCH_SIZE,)).to(device)

    model.train()
    optimizer.zero_grad()

    logits = model(dummy_input)
    loss = criterion(logits, dummy_labels)

    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
    optimizer.step()

    print(f"Demostracion RNN completada en dispositivo: {device}")
    print(f"Dimension de salida (Logits): {logits.shape}")
    print(f"Perdida inicial (Loss): {loss.item():.4f}")

    if __name__ == "__main__":
    train_rnn_demo()
```

9.2 Conexión con las prácticas

Los notebooks pasan del modelado de secuencias con MLP a una RNN implementada desde cero para predicción del siguiente carácter y exponen la necesidad de controlar el gradiente.

9.3 Autoevaluación razonada

Pregunta

¿Qué información conserva el estado oculto? **Solución.** Una representación del contexto procesado hasta el paso temporal actual.

Pregunta

¿Qué es gradient clipping? **Solución.** Una limitación de la norma del gradiente para evitar actualizaciones excesivas, especialmente útil frente a explosión del gradiente.

10 Atención y Transformers

10.1 Apuntes del Tema 11: Attention Mechanisms and Transformers

10.1.1 1. Introducción y Limitaciones de los Modelos Secuenciales (Seq2Seq)

10.1.1.1 1.1 El Enfoque Seq2Seq Tradicional (RNNs)

- **Modelado Secuencial:** Las redes neuronales recurrentes (RNNs) procesan las secuencias token a token de forma secuencial.
- **Arquitectura Encoder-Decoder:** En tareas como la traducción automática (p. ej., de inglés a español), el *Encoder* procesa la frase de entrada y condensa su información en un vector de estado oculto final h_{T_x}. Luego, el *Decoder* genera la secuencia traducida token a token condicionado en este vector.

10.1.1.2 1.2 El Cuello de Botella del Vector de Contexto (*Context Bottleneck*)

- **Limitación Principal:** Comprimir toda una frase de longitud variable en un único vector de tamaño fijo h_{T_x} genera una gran pérdida de información.
- **Fallo en Secuencias Largas:** Aunque funciona bien para frases cortas (p. ej., *"The black cat"* $\rightarrow$ *"El gato negro"*), el modelo falla en oraciones largas y complejas.
- **Dependencias a Largo Plazo:** Las palabras iniciales se "olvidan" a medida que la secuencia crece. Además, no gestiona eficientemente el reordenamiento de palabras entre idiomas.

10.1.2 2. Mecanismos de Atención en RNNs (Bahdanau et al., 2014)

10.1.2.1 2.1 Filosofía de la Atención

En lugar de depender de un único vector fijo, la atención permite al *Decoder* acceder a **todos** los estados ocultos del *Encoder* $\{h_1, h_2, \ldots, h_{T_x}\}$ en cada paso de decodificación.

- **Adaptabilidad:** En cada paso t, el *Decoder* calcula qué partes de la frase de entrada son más relevantes para predecir el siguiente token.
- **Vector de Contexto Dinámico (c_t):** Se calcula como una suma ponderada de todos los estados del *Encoder*:

$$c_t = \sum_{i=1}^{T_x} \alpha_{t,i} h_i$$

10.1.2.2 2.2 Formulación Matemática de la Atención de Bahdanau

Para cada paso de tiempo del decodificador i con estado anterior s_{i-1}:

1. **Puntuación de Alineamiento (*Score*):**

$$e_{ij} = a(s_{i-1}, h_j)$$

2. **Pesos de Atención (Softmax):**

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

3. **Vector de Contexto:**

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

4. **Actualización del Estado y Predicción:**

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

$$P(y_i \mid y_{1:i-1}, x) = g(y_{i-1}, s_i, c_i)$$

10.1.2.3 2.3 Inconvenientes de la Atención sobre RNNs

- **Dependencia Recurrente:** La computación sigue siendo secuencial (paso a paso), lo que impide la paralelizabilidad masiva en GPUs durante el entrenamiento.
- **Poco Directo:** La atención solo conecta el *Decoder* con el *Encoder*, pero la información dentro de cada bloque sigue fluyendo de forma secuencial implícita.

10.1.3 3. Autoatención (*Self-Attention*) en Transformers

Propuesto por Vaswani et al. (2017) en *"Attention is All You Need"*, elimina por completo la recurrencia. Cada palabra puede atender directamente a cualquier otra palabra de la secuencia sin importar la distancia.

10.1.3.1 3.1 Queries, Keys y Values (Q, K, V)

Para cada token con embedding $x \in \mathbb{R}^d$, se calculan tres proyecciones mediante matrices de pesos entrenables:

1. **Query ($q = W_Q x \in \mathbb{R}^{d_k}$):** Lo que el token busca saber sobre otros.
2. **Key ($k = W_K x \in \mathbb{R}^{d_k}$):** Cómo se identifica o etiqueta el token ante los demás.
3. **Value ($v = W_V x \in \mathbb{R}^{d_v}$):** La información que el token lleva para compartir.

10.1.3.2 3.2 Formulación Matricial: Scaled Dot-Product Attention

Dada la matriz de entrada $X \in \mathbb{R}^{T_x \times d_{\text{model}}}$, se proyectan las matrices $Q = XW_Q$, $K = XW_K$ y $V = XW_V$.

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- **Matriz de Similitud ($S = \frac{QK^\top}{\sqrt{d_k}}$):** Mide la afinidad entre las consultas y las claves.
- **Factor de Escala ($\sqrt{d_k}$):** Previene que el producto escalar alcance magnitudes muy grandes que saturarían la función Softmax (provocando gradientes nulos).

10.1.4 4. Atención Multicabeza (*Multi-Head Self-Attention*)

10.1.4.1 4.1 Motivación

Una sola cabeza de atención solo proyecta en un espacio único y puede capturar un único tipo de relación (p. ej., sujeto-verbo). El lenguaje requiere capturar múltiples relaciones simultáneamente (sintácticas, semánticas, modificadores).

10.1.4.2 4.2 Ecuaciones de Multi-Head Attention

Se entrenan h conjuntos independientes de matrices de proyección $\{W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}\}$:

$$\text{head}_i = \text{Attention}\left(Q^{(i)}, K^{(i)}, V^{(i)}\right) = \text{Attention}\left(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)}\right)$$

Las salidas de todas las cabezas se concatenan y se proyectan linealmente con $W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ para mantener las dimensiones de entrada:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)W_O$$

10.1.4.3 4.3 Dimensiones de Tensores en Práctica (Batching)

- Entrada $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ (B: Batch size, T: Longitud de secuencia).
- Proyección y cambio de forma para procesamiento paralelo:

$$Q, K, V \to \mathbb{R}^{B \times h \times T \times d_k}$$

- Salida de la atención por cabeza: $\mathbb{R}^{B \times h \times T \times d_v}$.
- Concatenación y proyección final: $\mathbb{R}^{B \times T \times d_{\text{model}}}$.

10.1.5 5. Arquitectura del Transformer: Encoder

El *Encoder* mapea la secuencia de entrada en una representación contextualizada manteniendo la dimensión del eje temporal T y la dimensión del embedding d_{model}. Consta de un apilamiento de N capas idénticas ($N = 6$ en la arquitectura original).

10.1.5.1 5.1 Codificación Posicional (*Positional Encoding*)

Dado que la autoatención es invariante a permutaciones, se debe añadir explícitamente información de orden.

- **Incorporación:** Se suma directamente al embedding de entrada en el primer nivel: $z_i = x_i + p_{\text{pos}}$.
- **Sinusoidal Encoding (Original Transformer):**

$$PE_{(\text{pos},2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(\text{pos},2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

- **Learned Positional Embeddings (BERT, GPT):** Tabla de parámetros entrenables $P \in \mathbb{R}^{T_{\max} \times d_{\text{model}}}$.

10.1.5.2 5.2 Conexiones Residuales y Normalización por Capa (Add & Norm)

Se aplican alrededor de cada subcapa (atención y red feedforward):

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- **Residual Connections:** Añaden la entrada de la capa a la salida $(x + \text{Sublayer}(x))$, facilitando el flujo del gradiente.
- **Layer Normalization:** Normaliza a través de las características d_{model} independientemente para cada token.
- **Variantes de Orden:**
  - *Post-LN (Original):* $\text{LN}(x + \text{Sublayer}(x))$.
  - *Pre-LN (Moderno - GPT/BERT):* $x + \text{Sublayer}(\text{LN}(x))$, mejora la estabilidad en redes muy profundas.

10.1.5.3 5.3 Red Feedforward (FFN)

Se aplica de forma independiente a cada token (*position-wise*):

$$\text{FFN}(x) = \text{Activation}(xW_1 + b_1)W_2 + b_2$$

- Expande la dimensión intermedia a d_{ff} (típicamente $d_{ff} = 4 \times d_{\text{model}}$).
- **Funciones de Activación:** *ReLU* $(\max(0, x))$ o *GELU* $(x \cdot \Phi(x)$ en BERT/GPT).

10.1.6 6. Arquitectura del Transformer: Decoder

El *Decoder* genera la secuencia objetivo token a token en modo autorregresivo.

10.1.6.1 6.1 Masked Multi-Head Self-Attention (Atención Causal)

Para evitar que el modelo acceda a tokens futuros durante el entrenamiento, se aplica una **máscara causal** sobre la matriz de puntuaciones previa al Softmax:

$$\text{scores}_{\text{masked}} = \text{scores} + M, \quad \text{donde } M_{ij} = \begin{cases} 0 & \text{si } i \geq j \\ -\infty & \text{si } i < j \end{cases}$$

10.1.6.2 6.2 Cross-Attention (Atención Cruzada Encoder-Decoder)

Conecta la representación del *Encoder* con el *Decoder*:

- **Query (Q):** Proviene del *Decoder* ($X_{\text{dec}}W_Q$).
- **Key (K) y Value (V):** Provienen de la salida final del *Encoder* ($H_{\text{enc}}W_K$, $H_{\text{enc}}W_V$).

$$\text{CrossAttention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

10.1.6.3 6.3 Capas Finales y Proyección de Salida

1. La salida oculta del decodificador $H_{\text{dec}} \in \mathbb{R}^{B\times T_y \times d_{\text{model}}}$ pasa por una capa lineal que proyecta al tamaño del vocabulario V:

$$\text{logits} = H_{\text{dec}}W_{\text{vocab}}^\top + b$$

2. Se aplica **Softmax** para obtener la distribución de probabilidad $P(y_t \mid y_{<t}, x)$.

10.1.7 7. Consejos de Entrenamiento, Inferencia y Arquitecturas

10.1.7.1 7.1 Estrategias de Entrenamiento

- **Subword Tokenization:** Uso de algoritmos como BPE (Byte-Pair Encoding) o WordPiece para evitar la explosión de vocabulario y manejar palabras fuera del diccionario (OOV).
- **Padding Masking:** Enmascaramiento con $-\infty$ para evitar atender a posiciones de relleno (`<PAD>`).
- **Warm-up Scheduling:** Incremento gradual de la tasa de aprendizaje inicial seguido de un decaimiento.

10.1.7.2 7.2 Estrategias de Decodificación en Inferencia

- **Greedy Decoding:** Selecciona $y_t = \arg\max_v P(v)$. Rápido, pero puede caer en óptimos locales.
- **Beam Search:** Mantiene las k mejores hipótesis parciales acumulando probabilidades logarítmicas.
- **Temperature Scaling:** Ajusta los logits dividiéndolos por T antes del Softmax ($T < 1$ más determinista, $T > 1$ más diverso).

10.1.7.3 7.3 Evolución de las Arquitecturas Transformer

- **Encoder-Only (BERT):** Atención bidireccional para comprensión y clasificación.
- **Decoder-Only (GPT-3, LLaMA, Mistral):** Atención causal para generación autorregresiva.
- **Encoder-Decoder (T5, BART):** Arquitectura seq2seq original para traducción y resumen.

10.1.8 8. Práctica en Código (PyTorch)

A continuación se incluye una implementación ejecutable de un módulo de **Multi-Head Self-Attention** nativo con enmascaramiento causal opcional:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadSelfAttention(nn.Module):
def __init__(self, d_model=512, num_heads=8):
super(MultiHeadSelfAttention, self).__init__()
assert d_model % num_heads == 0, "d_model debe ser divisible por num_heads"

self.d_model = d_model
self.num_heads = num_heads
self.d_k = d_model // num_heads

self.W_q = nn.Linear(d_model, d_model, bias=False)
self.W_k = nn.Linear(d_model, d_model, bias=False)
self.W_v = nn.Linear(d_model, d_model, bias=False)
self.W_o = nn.Linear(d_model, d_model, bias=False)

def forward(self, x, mask=None):
batch_size, seq_len, _ = x.size()

Q = self.W_q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1,
2)
K = self.W_k(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1,
2)
V = self.W_v(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1,
2)

scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)

if mask is not None:
scores = scores.masked_fill(mask == 0, -1e9)

attn_weights = F.softmax(scores, dim=-1)
attn_output = torch.matmul(attn_weights, V)

attn_output = attn_output.transpose(1, 2).contiguous().view(batch_size, seq_len,
self.d_model)
output = self.W_o(attn_output)

return output

if __name__ == "__main__":
B, T, d_model = 2, 5, 64
x = torch.randn(B, T, d_model)

causal_mask = torch.tril(torch.ones(T, T)).unsqueeze(0).unsqueeze(0)

mha = MultiHeadSelfAttention(d_model=d_model, num_heads=4)
out = mha(x, mask=causal_mask)
print(f"Dimension del tensor de entrada: {x.shape}")
print(f"Dimension del tensor procesado: {out.shape}")
```

10.2 Conexión con las prácticas

Las prácticas implementan atención, multi-head attention, codificaciones posicionales, AddNorm, FFN y un encoder Transformer; los tensores de prueba y las máscaras muestran el flujo de dimensiones.

10.3 Autoevaluación razonada

Pregunta

¿Por qué se escala el producto $QK^\top$? **Solución.** Para controlar su magnitud antes de softmax y evitar saturación cuando crece la dimensión de las claves.

Pregunta

¿Qué aporta la codificación posicional? **Solución.** Añade información sobre el orden, que no estaría determinada por la atención sobre un conjunto de tokens sin posiciones.

11 Sistemas de recomendación

Este capítulo se construye a partir de las fuentes de diapositivas y notebook indicadas al final.

11.1 Conexión con las prácticas

Este tema no tiene una nota Markdown equivalente en el repositorio; se incorpora como capítulo documentado desde sus diapositivas y notebook, manteniendo la trazabilidad de la fuente.

11.1.1 Fuente práctica: `13_01_Recommender_systems.ipynb`

Ruta: code/13_01_Recommender_systems/13_01_Recommender_systems.ipynb

Apartados detectados: 13_01 Recommender systems; Exercise 1 — Dataset: MovieLens 100K, Download & Files; Exercise 2 — Train/Validation/Test Partition (Leave-Last-N per User); Exercise 3 — Content-Based Recommender; Exercise 4 — Ranking Metrics (Precision@K, Recall@K, NDCG@K).

Salidas gráficas almacenadas: 0.

Salida textual conservada:

> Users: 943, Items: 1682, Genres: 19 | Exercise 1: all checks passed | train (98114, 6) unique users 943 unique items 1678 | valid (943, 6) unique users 943 unique items 534

Fragmento representativo:

```

# !pip install torch pandas numpy scikit-learn requests

import copy
import math
import random
import zipfile
import requests
import numpy as np
import pandas as pd

from pathlib import Path
from typing import Callable, Dict, List, Tuple

from sklearn.preprocessing import normalize

import torch
import torch.nn as nn
```

```
from torch.utils.data import Dataset, DataLoader, RandomSampler

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = 'cuda' if torch.cuda.is_available() else 'cpu'

NUM_GENRES = 19
K = 10
```

11.2 Autoevaluación razonada

Pregunta

¿Qué diferencia hay entre filtrado colaborativo y basado en contenido?

Solución. El colaborativo usa patrones de interacción entre usuarios e ítems; el basado en contenido usa atributos de los ítems y preferencias observadas.

Pregunta

¿Por qué una métrica de recomendación debe definirse con cuidado?

Solución. Porque precisión, ranking, cobertura y diversidad miden objetivos diferentes y pueden entrar en conflicto.